Хеширование и безопасное хранение паролей

Argon2 в Express + Prisma: только необходимое для REST API

Зачем это нужно в курсовой

- Если в REST API есть регистрация пользователей, пароли нужно хранить безопасно.
- При регистрации сохраняем хеш, а не исходный пароль.
- При входе проверяем пароль по сохранённому хешу.
- Никогда не отправляем пароль или его хеш в JSON-ответе.

Что научимся делать

- Объяснять, почему нельзя хранить пароль открытым текстом.
- Отличать хеширование от шифрования.
- Использовать argon2.hash() и argon2.verify().
- Проверять хранение хеша в PostgreSQL через Prisma.
- Понимать работу существующего passwordService из лабораторной №1.

Что будет при утечке таблицы users

Хранение	Что увидит посторонний	Оценка
password = Example123!	Исходный пароль	Недопустимо
password_hash = $argon2id$...	Хеш, а не исходный пароль	Правильный подход

Хеширование и шифрование — не одно и то же

Хеширование

- Одностороннее преобразование.
- При login используем verify().
- Исходный пароль из хеша не расшифровываем.

Шифрование

- Преобразование с возможностью расшифровки при наличии ключа.
- Подходит для данных, которые нужно восстановить.
- Не является основным способом хранения паролей.

Почему не обычный SHA-256?

- SHA-256 специально предназначен для быстрого вычисления хеша.
- Для хранения паролей высокая скорость помогает массово подбирать варианты.
- Argon2 — специализированный алгоритм хеширования паролей.
- В курсовой работе не нужно реализовывать криптографию самостоятельно: используем библиотеку.

Что такое salt — простыми словами

- Salt — случайное значение, которое участвует в вычислении хеша.
- Библиотека argon2 создаёт новый salt при каждом вызове hash().
- Поэтому два одинаковых пароля обычно имеют разные сохранённые хеши.
- Salt не нужно придумывать, создавать или хранить в отдельной таблице.

Один пароль — разные хеши

- Два пользователя выбирают Example123!. Каждый вызов argon2.hash() создаёт новый salt, поэтому строки хеша отличаются. Оба пароля всё равно корректно проверяются через verify().
- hash1 !== hash2, но verify(hash1, password) и verify(hash2, password) возвращают true.

Salt: одинаковый password → разные Argon2id hashes



User A

Один и тот же password

Password:

Example123!

salt A (случайный): `f3e9a7c2b1d4e8f9` (16 байт, случайное значение)

Argon2id
(password + salt + параметры)

Результат (hash A):

```
$argon2id$v=19$m=65536,t=3,p=4$
f3e9a7c2b1d4e8f9$K7xQ9mZt4vHc2L1r
aB8nW5sY0dP3qR6uT1vE9kLm2N8pF4z
```

Одинаковый password + разный salt = разные hashes

Один и тот же пароль может давать разные результаты, потому что для каждого хеша используется свой случайный salt.



User B

Один и тот же password

Password:

Example123!

salt B (случайный): `9c6d2f8e4a1b7c3d` (16 байт, случайное значение)

Argon2id
(password + salt + параметры)

Результат (hash B):

```
$argon2id$v=19$m=65536,t=3,p=4$
9c6d2f8e4a1b7c3d$W4rT8pLm9zQ2vH6k
dN1sY7uF0aC3eR5tB8mK9xZ2qP6vL1n
```

Ключевые выводы:

- Salt — случайное уникальное значение для каждого password hash.
- Даже одинаковые passwords не дают одинаковый сохранённый hash.
- Salt хранится вместе с hash и не является секретом.
- Это защищает от rainbow tables и упрощённого массового сравнения hashes.

Данные в базе (например, таблица users)

id	username	passwordHash
1	user_a	$argon2id$v=19$m=65536,t=3,p=4$f3e9a7c2b1d4e8f9$K7xQ9mZt...
2	user_b	$argon2id$v=19$m=65536,t=3,p=4$9c6d2f8e4a1b7c3d$W4rT8pLm...

В базе хранятся уже готовые хэши (вместе с salt и параметрами Argon2id).

Почему выбираем Argon2

- Argon2id предназначен для безопасного хеширования паролей.
- Алгоритм делает массовую проверку паролей затратной по времени и памяти.
- Для учебного приложения используем проверенную библиотеку argon2 с её настройками по умолчанию.
- Параметры memoryCost, timeCost и parallelism сегодня подробно не настраиваем.

Пакет argon2: установка и импорт

```powershell
npm install argon2
```

Что важно

- В проекте из ЛР-1 пакет уже установлен.
- Повторная установка не требуется, если он есть в package.json.
- В JavaScript используется import * as argon2 from "argon2";.

Создание хеша: hash()

JAVASCRIPT

```
import * as argon2 from "argon2";

const password = "Example123!";
const passwordHash = await argon2.hash(password);

console.log(passwordHash);
```

Что важно

- hash() асинхронный — используем await.
- Результат — строка, содержащая данные для дальнейшей проверки.
- В PostgreSQL храним password_hash, а не password.

Проверка пароля: verify()

```javascript
const correct = await argon2.verify(
  passwordHash,
  "Example123!"
);

const wrong = await argon2.verify(
  passwordHash,
  "wrong-password"
);

console.log(correct); // true
console.log(wrong);   // false
```

Что важно

- Первый аргумент — сохранённый хеш.
- Второй аргумент — введённый пароль.
- Нельзя сравнивать заново вычисленный хеш с сохранённым через ===.

Один пароль. Два сценария. Одна цель — безопасность.

argon2.hash() при register и argon2.verify() при login

Безопасные пароли — более безопасный мир

Register



Пользователь регистрируется

→ •••••••• **password**
(вводит пользователь)

→ **argon2.hash()**
Создаёт хеш из пароля

→ $argon2id$v=19$m=6... **passwordHash**
(результат хеширования)

→ **PostgreSQL**
Сохраняется только passwordHash

✓ **В базе хранится только passwordHash**

Login



Пользователь входит в систему

→ •••••••• **password**
(вводит пользователь при login)

\+



stored passwordHash
(загружается из PostgreSQL)

→ **argon2.verify()**
Проверяет введённый пароль и сохранённый passwordHash

→ ✓ **true**
Пароль верный (можно войти)

✗ **false**
Пароль неверный (доступ запрещён)

Безопасность начинается с правильных принципов

🚫 **Login не создаёт новый hash для сравнения**

⚠ **При login используется verify(), а не новое hash()**

Проверяем, а не пересчитываем!

Мини-эксперимент: сравните два хеша

JAVASCRIPT

```
const password = "Example123!";
const hash1 = await argon2.hash(password);
const hash2 = await argon2.hash(password);

console.log(hash1 === hash2); // false
console.log(await argon2.verify(hash1, password)); // true
console.log(await argon2.verify(hash2, password)); // true
```

Что важно

- Одинаковый пароль не означает одинаковые строки хеша.
- verify() умеет проверить пароль, используя сведения внутри сохранённого хеша.
- Это главный новый эксперимент лабораторной №2.

Модель User в Prisma

```prisma
model User {
  id           Int    @id @default(autoincrement())
  name         String
  email        String @unique
  passwordHash String @map("password_hash")
  role         String @default("user")

  @@map("users")
}
```

Что важно

- User и users — модель Prisma и таблица PostgreSQL.
- passwordHash в JavaScript соответствует password_hash в БД.
- Модель уже создана в ЛР-1; повторно создавать миграцию не нужно.

Где уже находится логика хеширования

JAVASCRIPT

```
// services/passwordService.js (из ЛР-1)
import * as argon2 from "argon2";

export async function hashPassword(password) {
  return argon2.hash(password);
}

export async function verifyPassword(hash, password) {
  return argon2.verify(hash, password);
}
```

Что важно

- Файл уже существует после лабораторной №1.
- Изучаем назначение функций, а не переписываем тот же код.
- Другие сервисы вызывают эти две функции.

Что происходит при регистрации

- 1. Запрос — POST /api/auth/register: name, email, password
- 2. Проверка — Проверяем входные данные и уникальность email
- 3. Хеш — hashPassword(password) создаёт passwordHash
- 4. Сохранение — prisma.user.create() записывает passwordHash в PostgreSQL
- 5. Ответ — Возвращаем id, name, email, role — без passwordHash

Регистрация: главные строки

```javascript
const passwordHash = await hashPassword(password);

const user = await prisma.user.create({
  data: { name, email, passwordHash },
  select: { id: true, name: true, email: true, role: true }
});
```

Что важно

- Этот фрагмент — часть registerUser() из ЛР-1.
- Проверка данных и повторного email выполняется до него.
- select не позволяет вернуть хеш пользователю.

Что происходит при входе

- 1. Запрос — POST /api/auth/login: email, password
- 2. Поиск — prisma.user.findUnique({ where: { email } })
- 3. Проверка — verifyPassword(user.passwordHash, password)
- 4. Результат — true — вход успешен; false — ответ 401

Login: главные строки

```javascript
const user = await prisma.user.findUnique({
  where: { email }
});
if (!user) return null;

const valid = await verifyPassword(
  user.passwordHash, password
);
if (!valid) return null;
```

Что важно

- Неизвестный email и неверный пароль дают одинаковый внешний ответ.
- Новый хеш при входе не создаём.
- Проверка использует уже сохранённый passwordHash.

Полный путь password: register → hash → DB → login → verify

От регистрации пользователя до последующей проверки пароля. Plaintext пароль нигде не хранится.

Безопасное хранение и проверка паролей в Express Internet Shop

1 Register (регистрация)

Пользователь создаёт аккаунт

Создание аккаунта

user@example.com

••••••••

Зарегистрироваться

2 Auth Controller

Принимает данные (email, password)

```
// authController.register()
const { email, password } = req.body;
await authService.register({
  email,
  password
});
```

3 Auth Service

Хеширует пароль

```
// authService.register()
const passwordHash = await
  argon2.hash(password, {
    type: argon2.argon2id,
    memoryCost: 2 ** 16,
    timeCost: 3,
    parallelism: 1
  });
await prisma.user.create({
  data: { email, passwordHash }
});
```

4 Prisma Client

Сохраняет в базу

```
// Prisma
await prisma.user.create({
  data: {
    email: "user@example.com",
    passwordHash
  }
});
```

5 PostgreSQL

В базе хранится хеш

users

id	UUID
email	VARCHAR
passwordHash	VARCHAR
name	VARCHAR
role	VARCHAR
createdAt	TIMESTAMP

Пользователь успешно зарегистрирован!

В базе хранится только хеш пароля, а не сам пароль.

1 Login (вход)

Пользователь вводит свои данные

Вход в аккаунт

user@example.com

••••••••

Войти

Нет аккаунта? Регистрация

2 Auth Controller

Передаёт данные в сервис

```
// authController.login()
const { email, password } = req.body;
const user = await authService.login({
  email,
  password
});
```

3 Auth Service

Проверяет пароль

```
// authService.login()
const user = await prisma.user
  .findUnique({ where: { email } });
if (!user) throw new Error('Invalid');
const isValid = await argon2.verify(
  user.passwordHash,
  password
);
if (!isValid) throw new Error('Invalid');
return user;
```

4 Prisma Client

Ищет пользователя

```
// Prisma
const user = await prisma.user
  .findUnique({
    where: { email },
    select: {
      id: true,
      email: true,
      passwordHash: true,
      name: true,
      role: true
    }
});
```

5 Сравнение хеша

argon2.verify()

Введённый пароль (plaintext): MyP@ssw0rd

Stored passwordHash (из базы): $argon2id$v=19$m=...

argon2.verify()

true

Пароль верный
Пользователь авторизован

false

Неверный пароль
Возвращается ошибка 401

При входе НЕ создаётся новый хеш!

Введённый пароль проверяется через argon2.verify() с сохранённым хешем.

Ключевые принципы

- Plaintext пароль нигде не хранится
- Используйте Argon2id (рекомендуется OWASP)
- Уникальный salt генерируется автоматически
- Настройте параметры (memory, time, parallelism)

Рекомендуемые параметры Argon2id

- **memoryCost:** 2 ** 16 (64 MiB)
- **timeCost:** 2–3
- **parallelism:** 1
- Используйте библиотеку "argon2" для Node.js

Результат

- Безопасное хранение паролей
- Устойчивая защита от перебора (brute force)
- Соответствие современным рекомендациям (OWASP)
- Готово к использованию в Express Internet Shop

Express Internet Shop

Безопасно.
Современно.
Надёжно.

Какие данные можно возвращать через API?

Поле	Вернуть клиенту?	Почему
id, name, email, role	Да	Нужны приложению
password	Нет	Секрет пользователя
passwordHash	Нет	Секретное служебное значение

Как проверить реализацию в своём проекте

- Регистрация: новый пользователь появляется в таблице users.
- В pgAdmin поле password_hash заполнено, исходного пароля в таблице нет.
- Login с верным паролем успешен.
- Login с ошибочным паролем возвращает 401.
- Оба ответа API не содержат passwordHash.

Типичные ошибки новичков

- Сохраняют пароль в отдельном поле password — так делать нельзя.
- Сравнивают hash() с прежним хешем через === — используют verify().
- Возвращают результат prisma.user.findUnique() целиком в HTTP-ответе.
- Выводят пароли или хеши в обычные журналы сервера.
- Заново создают модель и сервис, хотя они уже готовы после ЛР-1.

Дополнительно для курсовой: смена пароля

- Приложение получает старый и новый пароль.
- Сначала проверяет старый пароль через verifyPassword().
- Создаёт хеш нового пароля через hashPassword().
- Сохраняет новый passwordHash в users.
- После изменения старый пароль больше не должен подходить.
- Это расширение задания; полноценное управление доступом будет изучено дальше.

Архитектура безопасного password storage в Internet Shop

Разделение ответственности, безопасное хеширование паролей и защита данных пользователей

Frontend (Browser)

React / HTML + JS

Регистрация (register)

Создать аккаунт

user@example.com

••••••••

Виктор

Зарегистрироваться

Вход (login)

Вход в аккаунт

user@example.com

••••••••

Войти

POST /api/auth/register

```
{
  "email": "...",
  "password": "...",
  "name": "..."
}
```

POST /api/auth/login

```
{
  "email": "...",
  "password": "..."
}
```

Express.js Backend

express

Auth Routes

Маршруты

```
// routes/auth.js

router.post(
  '/register',
  authController.register
);

router.post(
  '/login',
  authController.login
);
```

Auth Controller

Обработка запроса

register(req, res)

- Валидация данных
- Вызов authService register()
- Формирование ответа

login(req, res)

- Валидация данных
- Вызов authService login()
- Формирование ответа

Auth Service

Бизнес-логика

```
async register(data) {
  const passwordHash =
   await passwordService.
   hash(data.password);
return prisma.user.create({
  data: {
    email: data.email,
    passwordHash,
    name: data.name
  });
}
```

```
async login(email, password) {
  const user = await prisma.
  user.findUnique({ where:{
  email } });
const isValid = await
  passwordService.verify(
  password, user.passwordHash
  );
if (!isValid) throw new Error
  ('Invalid credentials');
  return user;
}
```

Password Service

Хеширование и проверка

```
import argon2 from 'argon2';

export const hash = async
(password) => {
  return await argon2.hash(
    password, {
      type: argon2.argon2id,
      memoryCost: 2 ** 16,
      timeCost: 3,
      parallelism: 1
  });
};
```

```
export const verify = async
(password, hash) => {
  return await argon2.verify(
  hash, password);
};
```

Prisma Client

Доступ к БД

```
import { PrismaClient }
from '@prisma/client';

const prisma = new
PrismaClient();
```

PostgreSQL

База данных

Таблица users

id	UUID
email	VARCHAR
passwordHash	VARCHAR
name	VARCHAR
role	VARCHAR
createdAt	TIMESTAMP
updatedAt	TIMESTAMP

В базе хранится только **passwordHash**, а не сам пароль!

201 Created

```
{
  "user": {
    "id": "...",
    "email": "user@example.com",
    "name": "Виктор",
    "role": "user"
  }
}
```

200 OK

```
{
  "user": {
    "id": "...",
    "email": "user@example.com",
    "name": "Виктор",
    "role": "user"
  },
  "token": "<JWT>"
}
```

Пароль никогда не возвращается клиенту!

Ключевые преимущества

- Используется современный алгоритм хеширования (Argon2id)
- Уникальный salt генерируется автоматически
- Пароль нигде не хранится в открытом виде
- В базе хранится только passwordHash
- При входе пароль проверяется через argon2.verify()
- Чёткое разделение слоёв (Controller → Service → Prisma)
- Готово к расширению (JWT, роли, защита маршрутов)

Рекомендации

- Используйте Argon2id (рекомендуется OWASP)
- Параметры: memoryCost ≥ 2**16 (64 MiB), timeCost 2–3, parallelism 1
- Используйте HTTPS
- Добавьте rate limiting для /login
- Логируйте неудачные попытки входа
- Не возвращайте passwordHash в API-ответах
- Используйте requireAuth для защищённых маршрутов

Итоги

1. Никогда не сохраняем исходный пароль.
2. argon2.hash() — при регистрации или смене пароля.
3. argon2.verify() — при проверке введённого пароля.
4. Различающиеся хеши одинакового пароля — нормальное поведение.
5. В PostgreSQL хранится password_hash; в HTTP-ответе его нет.

Контрольные вопросы

1. Почему нельзя хранить password открытым текстом?
2. Чем хеширование отличается от шифрования?
3. Почему два хеша одного пароля отличаются?
4. Чем отличаются argon2.hash() и argon2.verify()?
5. Почему сравнение hash1 === hash2 не подходит для проверки пароля?
6. Какие поля пользователя запрещено возвращать в JSON-ответе?